

SI 100B Programming Assignment 2

SI 100B TA Team

October 17, 2025

Notice

- This homework is due **21:00 October 23, Thursday**, please start early.
- The problems should be solved **individually**. You should submit solutions for all problems to the **online judge** (OJ) system, which will give a score for each submission automatically.
- You may discuss course material, programming languages, general techniques, and share test cases with others.
- You're encouraged to ask any question about problem description, environment setup, error information, grammar, functions, methods etc., or post general ideas at Piazza. It can help others too!
- However, you may **NOT** read, possess, copy, or submit any part of other's solution. You may not ask others to directly debug your program or provide specific lines of solution. You should not access another person's account. Also, do not offer such help, and protect your account and code. Giving or receiving inappropriate help is **plagiarism**, an infringement of academic integrity.
- You are not allowed to write your solution directly by or based on code generated by **AI tools**.
- **Good luck and have fun!**

1 Calculator with lambda functions (25 points)

1.1 Task

Implement a function `calc(op, x, y)`, where `op` is a binary operation function. The function should return the result of `op(x, y)`.

Also, implement a helper function `get_op(op_name)`:

Given a string `op_name` (such as `"add"` or `"mul"`), return the corresponding **lambda function**. You need to implement the following binary operations using **lambda expressions**:

- `add`: addition, $x + y$
- `sub`: subtraction, $x - y$
- `mul`: multiplication, $x * y$
- `div`: division, x / y
- `pow`: exponentiation, x^y
- `max`: return the greater one of the two values
- `min`: return the smaller one of the two values

1.2 Submission

Please submit **only your** `calc` and `get_op` functions. Do **not** submit any testing or print statements.

1.3 Hint

If you can't get points on OJ, you can test in the ways below:

- **Verify that your functions return the correct value types.**
- Test your implementation locally with sample cases, for example:

```
print(get_op('add')(2, 3))  
calc(get_op('add'), 2, 3) == 5
```

to check your functions.
- Make sure you only **submit** the two required functions and no additional testing code.

2 Image Linear Enhancement (25 points)

In 8-bit grayscale imaging, each pixel is an integer in $[0, 255]$. A common global enhancement is an affine (linear + bias) transform:

$$y = a \cdot x + b$$

where x is the input pixel, a adjusts contrast (gain), and b shifts brightness (bias). Implementations must also quantize to integers and clip results to a target range so the output stays valid for display or storage.

2.1 Task

Given:

- a list of pixel values, each in the range $[0, 255]$;
- real coefficients a and b ;
- an output dynamic range $[low, high]$, which satisfies $0 \leq low \leq high \leq 255$.

For each input pixel x , perform the following steps:

Linear transform — compute the adjusted pixel value $y = a \cdot x + b$.

Quantization — convert y to an integer using the Half-away-from-zero rounding rule:

For $y \geq 0$:

$$y_q = \begin{cases} \lfloor y \rfloor, & \text{if } y - \lfloor y \rfloor < 0.5, \\ \lceil y \rceil, & \text{if } y - \lfloor y \rfloor \geq 0.5. \end{cases}$$

Clipping — ensure the quantized result stays within the valid range $[low, high]$; any values below low are set to low , and values above $high$ are set to $high$.

Return the list of enhanced pixels as integers within $[low, high]$.

2.2 Input/Output Specification

2.2.1 Input

Four lines:

An integer n represents number of pixels

n integers, each in $[0, 255]$

Two reals: a, b

Two integers: $low, high$ (satisfy $0 \leq low \leq high \leq 255$)

$1 \leq n \leq 100, -1000 \leq a, b \leq 1000$.

2.2.2 Output

One line with n integers (the enhanced pixels), separated by single spaces. No trailing spaces.

2.2.3 Example

1. Input

```
3
10 100 230
1.2 -10
0 255
```

Output

```
2 110 255
```

3 Simplified Match-3 Game (25 points)

You are developing a simplified match-3 puzzle game. The game is played on an $n \times m$ grid where each cell contains a colored block represented by a capital letter. This assignment focuses on implementing basic 2D list operations and simple game mechanics.

3.1 Task

Implement a simplified version of a match-3 game with the following rules:

1. Game Board:

- The game is played on an $n \times m$ grid ($3 \leq n, m \leq 10$).
- Each cell contains an uppercase letter (A-Z) representing a colored block.
- **The initial board will never contain any pre-existing matches.**

2. Game Commands:

- `swap x1 y1 x2 y2`:
 - Swaps the blocks at positions $(x1, y1)$ and $(x2, y2)$, where x stands for rows (the input n), and y stands for columns (the input m).
 - Coordinates are 1-indexed (top-left corner is $(1, 1)$).
- `exit`:
 - Ends the game immediately.
 - Outputs final score before terminating.

3. Valid Moves:

- Players can only swap two adjacent blocks (up/down/left/right).
- Diagonal swaps are not allowed.

4. Matching Rules:

- A match occurs when exactly 3 identical blocks line up horizontally or vertically.
- **Special Rule:** As a simplified match-3 game, the match only counts if the swap places a block in the middle position of the 3-in-a-row, **i.e. the swapped block must become the center of 3 identical blocks.**
- Each valid match awards exactly 30 points.

5. Match Resolution:

- All 3 matched blocks are removed from the board.
- **The empty spaces are filled with ' ' characters.**
- No blocks will fall down to fill empty spaces.
- No chain reactions occur after a match.

6. Invalid Moves:

- Any swap that doesn't create a valid match (as defined above).
 - Attempting to swap non-adjacent blocks.
 - Trying to swap blocks outside the grid boundaries.
- **All invalid swaps will be automatically undone.**

3.2 Input/Output Specification**3.2.1 Input**

1. First line: Two integers n and m ($3 \leq n, m \leq 10$), representing grid dimensions.
2. Next n lines: m space-separated uppercase letters (A-Z) representing initial grid.
3. Subsequent lines: Game commands:
 - `swap x1 y1 x2 y2` to swap blocks at positions $(x1, y1)$ and $(x2, y2)$.
 - `exit` to end game.

3.2.2 Output

- To start game, print the current grid and current score.
- After that, for each swap operation print:
 1. Current grid state (n lines of m space-separated characters).
 2. Current score.
- After “`exit`” command print: Final score.

3.2.3 Example**1. Input 1**

```

3 3
A B C
B C A
C B A
swap 1 1 2 1
swap 2 1 2 2
swap 1 1 3 3
exit

```

2. Output 1

```

A B C
B C A
C B A
0
A B C
B C A
C B A
0
A . C
C . A
C . A
30
A . C
C . A
C . A
30
30

```

3.2.4 Hint.

The emptied block '.' should not cause a valid swap.

4 F1 Race Strategy Analysis (25 points)

4.1 Background

You are an F1 race strategy engineer. During an intense race, you need to analyze telemetry data sent from your driver in real time to make the best strategy decisions such as determining pit-stop windows or evaluating tire performance. You will receive all lap time data of a driver in one race. Each lap contains **three** sector times (Sector 1, Sector 2, Sector 3) and the **pit-stop** time at the end of the lap. Your task is to process this raw data and respond to a series of strategy team queries.

4.2 Definitions

4.2.1 Lap Time

The pure track time of a lap:

$$\text{lap_time} = S1 + S2 + S3$$

4.2.2 Total Time

The total time from the end of the previous lap to the end of the current lap:

$$\text{total_time} = S1 + S2 + S3 + \text{pit_stop_time}$$

If there is **no pit stop**, pit stop time will be **0.0**.

4.2.3 Sector Colors

When processing the lap data in sequence, you should keep track of the personal best time (PB) for each of the three sectors (S1, S2, S3). A personal best means the **fastest** time the driver has achieved in that specific sector so far.

For each new lap:

- If the sector time is smaller (**less**) than the previous PB, mark this sector as **Green**. This means the driver has just set a new record for that sector.
- If the sector time is **bigger or equal** to the previous PB, mark this sector as **Yellow**. This means the driver did not improve in that sector.

Note: The attributes listed above are refreshed whenever a new line of data is processed. (Especially sector color) You need to calculate the attributes above separately for each lap.

4.3 Format

4.3.1 Input

An **integer** `L` represents total number of laps at first line. The next `L` lines: Each lap's data in the format:

```
lap_number, s1_time, s2_time, s3_time, pit_stop_time
```

where:

- `lap_number`: integer (starting from 0)
- `s1_time`, `s2_time`, `s3_time`: floating numbers
- `pit_stop_time`: floating number; 0.0 if no pit stop

An **integer** `N` represents number of query commands at the next line. The following `N` lines: Each contains one query instruction (see below).

4.3.2 Output

For each query, print **one line** of result. All floating values must be rounded to **two decimal places**.

4.4 Query Instructions

Command	Description	Output Type
<code>best_lap</code>	Returns the lap number with the fastest lap time. (If tied, return the smallest lap number.)	Integer
<code>pit_laps</code>	Returns a list of all lap numbers where a pit stop occurred.	List
<code>lap_time, i</code>	Returns the lap time of Lap i .	Float
<code>best_sector, s</code>	Returns the all-time personal best time for Sector s .	Float
<code>pit_time, t</code>	Returns a list of lap numbers where a pit stop occurred with time $\leq t$ (ascending order).	List
<code>avg_lap, start, end, step</code>	Returns the average lap time of laps within the slice <code>[start:end:step]</code> .	Float
<code>stint_time, start, end, step</code>	Returns the sum of total time for laps in the slice.	Float
<code>green_laps, s, start, end, step</code>	Returns the list of laps where Sector s was green within the slice (ascending order).	List
<code>pit_rate, start, end, step</code>	Returns the pit-stop rate = (number of pit-stop laps) $\div$ (slice length).	Float
<code>reorder, key, ASC/DESC</code>	Reorders the current lap dataset by <code>key</code> in ascending (<code>ASC</code>) or descending (<code>DESC</code>) order. (Ties resolved by lap number ascending.) This command has no output . Possible keys: <code>,</code> , <code>s1</code> , <code>s2</code> , <code>s3</code> .	None

Note: In all multi-parameter queries, `start`, `end`, and `step` refer to **index positions**. You should first perform slicing on the lap dataset according to these indices (`[start,end]`). All three values are guaranteed to be integers. If the resulting slice is empty, output: `Querying on empty list!` and do nothing else.

4.5 Samples

4.5.1 Sample 1

Input 1

```
4
0,30,30,30,2.0
1,30,30,30,1.0
2,30,30,30,0
3,30,30,30,3.0
5
best_lap
pit_laps
```

```

pit_time,1.5
best_sector,1
lap_time,0

```

Output 1

```

0
[0, 1, 3]
[1]
30.00
90.00

```

4.5.2 Sample 2

Input 2

```

3
0,30,30,30,0
1,31,31,31,0
2,32,32,32,0
7
avg_lap,0,2,-1
best_lap
lap_time,2
pit_time,0.1
green_laps,1,2,0,-1
pit_rate,0,2,1
best_sector,1

```

Output 2

```

Querying on empty list!
0
96.00
[]
[]
0.00
30.00

```

4.6 Hint

We suggest you to write functions to handle a command's actual logics; it is **not recommended** to simply fill all commands' logics into a chunk of branches (or something similar).

Each `reorder` affects the order of lap data for subsequent queries.